

M12 - Distributed Processing with Spark & PySpark

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Concept before command

Every lesson explains the mental model and correctness/performance evidence before asking you to run the project.

Contents

- DE12-01** - Why Spark exists: driver, executors, partitions and tasks
- DE12-02** - PySpark setup, SparkSession and laptop-safe local mode
- DE12-03** - DataFrames and explicit schemas
- DE12-04** - Spark SQL and expression equivalence
- DE12-05** - Lazy transformations and actions
- DE12-06** - Jobs, stages, tasks and physical plans
- DE12-07** - repartition vs coalesce
- DE12-08** - Aggregations and shuffle cost
- DE12-09** - Joins and broadcast strategy
- DE12-10** - Join correctness, fan-out and reconciliation
- DE12-11** - Data skew
- DE12-12** - Adaptive Query Execution (AQE)
- DE12-13** - Caching and persistence
- DE12-14** - Parquet I/O and partition pruning
- DE12-15** - Built-in functions vs Python UDFs
- DE12-16** - Driver safety: collect, toPandas and bounded inspection
- DE12-17** - Testing and debugging Spark transformations
- DE12-18** - Integrated PySpark pipeline and distributed-processing checkpoint

DE12-01 - Why Spark exists: driver, executors, partitions and tasks

What you will be able to do

Explain why Spark distributes work and distinguish driver, executor, partition and task.

Why this matters

Large datasets or expensive transformations need parallel work across cores/machines while a coordinator plans the job.

Picture it this way

The driver is a site manager, executors are workers, partitions are chunks of material, and tasks are instructions assigned to workers for one chunk/stage.

Start from zero

- The **driver** runs your application logic/plans jobs and coordinates executors.
- Executors perform tasks and hold partition data/cache in cluster mode.
- A DataFrame/RDD is divided into partitions; one task typically processes one partition for a stage.
- More partitions can increase parallelism but also scheduling/shuffle/file overhead.
- Spark is not automatically faster for tiny data; distributed overhead can dominate.
- Keep large data distributed; bringing everything to driver defeats the model.

Teacher demonstration

```
# Conceptual
DataFrame partitions: [P0][P1][P2][P3]
Stage task set:      T0  T1  T2  T3
Driver schedules tasks; executors process partitions.
```

Read the example like English

- Partition is data division; task is unit of execution for a stage/partition.
- A job can contain several stages separated by shuffle boundaries.

Expected check

term	role
driver	plans/co-ordinates
executor	runs tasks/stores partition data
partition	data chunk
task	stage work on partition

Common beginner traps

- Calling executors separate databases.
- Increasing partitions endlessly.
- Using collect() to make all work local.

Now you do a similar task

Draw a four-partition job with driver + two executors and show which tasks could run in parallel. Explain why this is conceptual in local mode.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What is the difference between a partition and a task?

Answer in your own words before continuing.

DE12-02 - PySpark setup, SparkSession and laptop-safe local mode

What you will be able to do

Start a SparkSession in local mode safely and prove Spark/Java/Python environment before changing setup.

Why this matters

PySpark requires JVM/runtime integration; environment errors should be diagnosed, not solved by reinstalling random versions.

Picture it this way

Local mode is a miniature practice workshop using your laptop cores while keeping the same Spark programming model.

Start from zero

- Reuse the supplied project requirements; install pyspark in a module venv.
- SparkSession is the main entry point for DataFrame/SQL APIs.
- master local[2] limits the lab to two local worker threads; local[*] may use all available cores.
- Set small shuffle partitions/log level for laptop training where appropriate.
- Verify spark.version, master and a tiny range/count before running full project.
- Java compatibility is a runtime dependency; if the probe fails, record exact error/version before changing Java/Python.

Teacher demonstration

```
from pyspark.sql import SparkSession
spark = (SparkSession.builder
        .master("local[2]")
        .appName("m12-training")
        .getOrCreate())
print(spark.version)
print(spark.sparkContext.master)
print(spark.range(10).count())
spark.stop()
```

Read the example like English

- Session creation proves JVM gateway/runtime started.
- range(10).count() triggers a tiny action and should return 10.
- stop releases local Spark resources.

Expected check

check	expected
master	local[2] or documented local setting
tiny count	10
runtime	no unexplained Java gateway error

Common beginner traps

- Installing several PySpark/Java versions at once.
- Using local[*] on a constrained laptop without reason.
- Treating SparkSession creation as proof your transformations are correct.

Now you do a similar task

Run the supplied session/partition lab. Save spark version/master/default partitions and a tiny successful action.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What does local[2] change compared with a real cluster?

Answer in your own words before continuing.

DE12-03 - DataFrames and explicit schemas

What you will be able to do

Create DataFrames with explicit schema and understand Spark column expressions/types/nullability.

Why this matters

Schema inference can be convenient but unstable/expensive for production sources; wrong types propagate into distributed transformations.

Picture it this way

A schema is the blueprint each worker uses to interpret a field consistently before processing partitions.

Start from zero

- DataFrame is a distributed table-like dataset with named typed columns.
- Explicit StructType/DDDL schema avoids guessing and documents contract.
- String, integer, double/decimal, date/timestamp have different operations.
- Nullability in Spark schema is metadata/behavior hint, not a full database NOT NULL constraint; still validate data.
- Use select/withColumn/filter through Spark Column expressions rather than Python row loops.
- Inspect schema with printSchema and bounded show.

Teacher demonstration

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
schema=StructType([
    StructField("order_id", StringType(), False),
    StructField("qty", IntegerType(), True),
])
df=spark.createDataFrame([("01", 2), ("02", 3)], schema)
df.printSchema()
df.show()
```

Read the example like English

- Spark knows qty is integer, so arithmetic does not rely on per-row string parsing.
- false nullable metadata documents expected order_id presence but business validation is still needed.

Expected check

proof	expected
schema	explicit types
rows	2
qty operation	numeric expression works

Common beginner traps

- Inferring important schema from dirty CSV in every run.
- Treating nullable=False as a substitute for all data quality checks.
- Using for row in df.collect() to transform.

Now you do a similar task

Read the raw project CSV with an explicit schema (or inspect supplied lab), then compare to inferred schema and explain differences/risks.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is explicit schema valuable even when inference happens to guess correctly today?

Answer in your own words before continuing.

DE12-04 - Spark SQL and expression equivalence

What you will be able to do

Express equivalent transformations using DataFrame API and Spark SQL while understanding they compile into Spark plans.

Why this matters

Teams use both styles. The correctness/performance question is about the logical/physical plan, not API tribalism.

Picture it this way

DataFrame API and SQL are two languages for giving the same planner a recipe.

Start from zero

- Register a temp view to query a DataFrame with Spark SQL.
- DataFrame functions such as select/filter/groupBy build expressions/plans.
- Equivalent SQL/DataFrame logic can optimize similarly through Catalyst.
- Compare schema/rows/control totals and explain plans, not just syntax.
- Avoid string-building SQL with untrusted values; use controlled expressions/parameters where supported.
- Use whichever style improves clarity/team/project integration.

Teacher demonstration

```
from pyspark.sql import functions as F
api = df.filter(F.col("status")=="Completed").select("order_id", "qty")
df.createOrReplaceTempView("orders")
sql = spark.sql("SELECT order_id, qty FROM orders WHERE status='Completed'")
print(api.count(), sql.count())
```

Read the example like English

- Both build lazy logical plans.
- Count triggers actions.
- Equivalent result should match key/row totals.

Expected check

style	strength
DataFrame API	composable typed-ish Python expressions
Spark SQL	familiar declarative SQL
optimizer	plans both forms

Common beginner traps

- Assuming SQL is always faster than DataFrame API.
- Mixing Python string loops with distributed expressions.
- Only comparing row count when values differ.

Now you do a similar task

Implement one filter/derived gross_sales with both APIs and reconcile rows + sum. Compare explain output.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

If SQL and DataFrame API produce the same optimized plan, what should drive your style choice?

Answer in your own words before continuing.

DE12-05 - Lazy transformations and actions

What you will be able to do

Predict lazy transformation behavior and identify which actions trigger execution.

Why this matters

Spark builds a plan first so it can optimize multiple transformations before touching the full dataset.

Picture it this way

You write a shopping list of transformations; Spark goes to the store only when an action asks for a result.

Start from zero

- Transformations (select/filter/withColumn/join/groupBy) generally create new lazy DataFrames/plans.
- Actions such as count, collect, write, take/show (depending implementation) trigger execution to produce results.
- Creating a DataFrame chain may do little source I/O until an action.
- Multiple actions can recompute the same lineage unless cached/persisted/materialized.
- Lazy evaluation enables optimizer predicate/projection pushdown and plan simplification.
- Side-effecting Python code should not be assumed to execute once per transformation definition.

Teacher demonstration

```
df2=(df
      .filter("status = 'Completed'")
      .withColumn("gross_sales", F.col("qty")*F.col("unit_price")))
# No result requested yet.
df2.explain()
print(df2.count()) # action
```

Read the example like English

- explain shows planned work without needing the full result.
- count triggers a Spark job.
- A second unrelated action may recompute lineage unless reused/cached.

Expected check

operation	lazy/action
filter/select/withColumn	transformation
join/groupBy	transformation
count/write/collect	action

Common beginner traps

- Thinking each line immediately processes data.
- Adding print statements inside UDF/task and expecting driver-order execution.
- Triggering many count/show actions accidentally.

Now you do a similar task

Run the lazy/explain lab and predict how many actions/jobs your script triggers before you execute it. Then confirm via Spark UI/logs if available.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can two actions on the same uncached DataFrame repeat expensive work?

Answer in your own words before continuing.

DE12-06 - Jobs, stages, tasks and physical plans

What you will be able to do

Read Spark job/stage/task/physical plan structure and identify shuffle boundaries.

Why this matters

Distributed performance debugging starts with the plan: where data is exchanged, how many partitions/tasks, and which operators dominate.

Picture it this way

A job is a delivery request; stages are work phases separated by material transfers; tasks are workers processing each partition in a phase.

Start from zero

- An action creates one or more jobs.
- A job is split into stages around shuffle/exchange boundaries.
- Each stage runs tasks over its partitions.
- **explain** displays logical/optimized/physical plan; Exchange nodes commonly indicate shuffle.
- Narrow transformations can often be pipelined within a stage; wide transformations need data redistribution.
- Plans are evidence, but runtime metrics/skew/data size matter too.

Teacher demonstration

```
result = df.groupBy("region").agg(F.sum("gross_sales").alias("sales"))
result.explain("formatted")
result.show()
```

Read the example like English

- groupBy usually requires an Exchange/shuffle by grouping key.
- show triggers execution of the plan.
- The number of post-shuffle partitions can differ from input partitions and may be adapted by AQE.

Expected check

plan clue	meaning
Exchange	shuffle/data redistribution
BroadcastHashJoin	small side broadcast
FileScan parquet	Parquet source
PartitionFilters	partition pruning clues

Common beginner traps

- Calling every Spark action one stage.
- Optimizing code without reading plan.
- Assuming a shuffle is always bad; many correct aggregations need it.

Now you do a similar task

Inspect formatted plans for a filter and a groupBy. Mark which has a shuffle/exchange and explain why.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does grouping by region normally require rows from different input partitions to move?

Answer in your own words before continuing.

DE12-07 - repartition vs coalesce

What you will be able to do

Choose repartition or coalesce based on desired partition count/distribution and shuffle tradeoff.

Why this matters

Partition count affects parallelism, output file count and later shuffle cost.

Picture it this way

repartition is reshuffling people into balanced buses; coalesce is combining neighboring buses with less movement.

Start from zero

- **repartition(n, cols...)** can increase/decrease partitions and generally shuffles to redistribute.
- **coalesce(n)** is mainly for reducing partitions with less/no full shuffle and may produce imbalance.
- Repartition by a key can prepare distribution for some downstream operation/output, but creates immediate shuffle cost.
- Do not set partitions from row count alone; consider bytes, cluster cores, skew and downstream file size.
- Inspect `df.rdd.getNumPartitions()` and explain plan.
- Use `coalesce(1)` only for tiny demonstration output; single-partition production writes bottleneck large jobs.

Teacher demonstration

```
print(df.rdd.getNumPartitions())
a=df.repartition(4,"region")
b=a.coalesce(2)
print(a.rdd.getNumPartitions(), b.rdd.getNumPartitions())
a.explain()
```

Read the example like English

- repartition introduces Exchange.
- coalesce reduces to 2 without redistributing as fully, so balance may remain uneven.
- Actual runtime benefit depends on data/next operations.

Expected check

goal	typical tool
increase partitions	repartition
rebalance by key	repartition(key)
reduce cheaply	coalesce
one tiny demo file	coalesce(1), not general pattern

Common beginner traps

- `coalesce(1)` on large output.
- repartition before every step.
- Judging balance only by partition count.

Now you do a similar task

Run the repartition/coalesce lab and save partition counts + plan. Explain when coalesce could leave one partition much larger.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can coalesce(2) be cheaper but less balanced than repartition(2)?

Answer in your own words before continuing.

DE12-08 - Aggregations and shuffle cost

What you will be able to do

Recognize aggregation shuffle cost and design pre-filtering/partition/key choices from evidence.

Why this matters

groupBy over distributed data must bring equal keys together, which often creates heavy network/disk shuffle.

Picture it this way

To total sales by region, all receipts for East must meet somewhere even if they started in many boxes.

Start from zero

- GroupBy/aggregate by non-partitioned key usually needs shuffle.
- Filter/project before aggregation to reduce shuffled bytes when logically possible.
- Built-in aggregate functions allow Spark partial/local aggregation optimization before shuffle in many cases.
- High-cardinality grouping creates many result groups/state and may be expensive.
- Use explain/runtime metrics for Exchange/shuffle read/write/spill.
- Do not manually collect groups to Python; keep aggregation distributed.

Teacher demonstration

```
completed=df.filter(F.col("status")== "Completed")
result=(completed
    .withColumn("gross",F.col("qty")*F.col("unit_price"))
    .groupBy("region")
    .agg(F.sum("gross").alias("sales")))
result.explain("formatted")
```

Read the example like English

- Filter happens before groupBy in the logical chain, reducing invalid/cancelled rows.
- GroupBy requires redistribution by region unless already appropriately partitioned.
- Result grain is one row per region.

Expected check

proof	expected
result grain	region
shuffle	Exchange present
total sales	reconciles to source completed total

Common beginner traps

- Aggregating before filtering when not equivalent.
- Calling shuffle an error.
- Using Python dict/group after collect.

Now you do a similar task

Run the shuffle aggregation lab. Reconcile regional sum to total sales and identify Exchange in plan.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What is one logically safe way to reduce data volume before a shuffle?

Answer in your own words before continuing.

DE12-09 - Joins and broadcast strategy

What you will be able to do

Choose join strategy from table size/key/cardinality and understand broadcast joins for truly small dimensions.

Why this matters

A normal distributed join may shuffle both sides by key. Broadcasting a small dimension can avoid moving the large side.

Picture it this way

Instead of sending every big shipment to a central join point, copy a tiny lookup booklet to every worker.

Start from zero

- Spark can choose broadcast hash join when one side is below/advised threshold and join type supports it.
- **broadcast(df)** is an explicit hint; use only when dimension is genuinely small enough for executor memory.
- Large-large joins typically require shuffle/partitioning strategy.
- Join keys/types/nulls/cardinality matter for correctness before performance.
- Inspect plan for BroadcastHashJoin/Exchange.
- Broadcasting a large table can cause memory pressure/failure.

Teacher demonstration

```
from pyspark.sql.functions import broadcast
joined = orders.join(broadcast(customers), "customer_id", "left")
joined.explain("formatted")
```

Read the example like English

- Customers are a small dimension in training, so broadcast is reasonable.
- Orders avoid a corresponding large-side shuffle for the broadcast join.
- Left join preserves order rows; unmatched customers must still be measured.

Expected check

strategy	when
broadcast small dimension	small lookup + supported join
shuffle join	large-large or non-broadcast
left join check	row count/unmatched keys

Common beginner traps

- Broadcasting because "dimensions are always small".
- Tuning strategy before verifying join correctness.
- Ignoring unmatched/null keys.

Now you do a similar task

Run join/broadcast lab, save plan + before/after row counts/unmatched IDs. Remove broadcast hint and compare observed plan.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What resource can fail if you broadcast a table that is too large?

Answer in your own words before continuing.

DE12-10 - Join correctness, fan-out and reconciliation

What you will be able to do

Prove join correctness by grain/cardinality/reconciliation and detect fan-out before optimizing it.

Why this matters

Distributed joins can multiply rows exactly like SQL/pandas joins. Spark scale makes the mistake expensive.

Picture it this way

If one order matches two customer-master rows, the factory produces two copies of that order; faster multiplication is still wrong.

Start from zero

- State left and right grain/uniqueness before join.
- Use duplicate-key checks/countDistinct/group counts on supposed dimension keys.
- Compare before/after row count when cardinality predicts preservation.
- Measure unmatched keys separately.
- Reconcile measures before/after when enrichment should preserve them.
- A legitimate one-to-many join changes grain; document expected multiplication rather than forcing row preservation.

Teacher demonstration

```
dups=customers.groupBy("customer_id").count().filter("count > 1")
assert dups.count()==0
before=orders.count()
joined=orders.join(customers,"customer_id","left")
after=joined.count()
print(before,after)
```

Read the example like English

- Dimension duplicate check establishes many-to-one expectation.
- If every order matches at most one customer, left join should preserve order count.
- Unmatched left rows can still remain with null dimension attributes.

Expected check

check	why
dimension key unique	prevents accidental fan-out
before/after rows	cardinality evidence
unmatched keys	relationship quality
measure total	no hidden multiplication

Common beginner traps

- Only checking Spark plan/performance.
- dropDuplicates after a fan-out instead of fixing the join model.
- Assuming customer_id unique because table is named dimension.

Now you do a similar task

Introduce/inspect a duplicate dimension key in a disposable DataFrame and demonstrate the fan-out, then repair upstream uniqueness and reconcile.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is dropDuplicates after a bad join not a trustworthy repair?

Answer in your own words before continuing.

DE12-11 - Data skew

What you will be able to do

Recognize data skew and explain why equal partition count can still produce one slow task.

Why this matters

If one key owns a huge share of rows, shuffle sends much of the data to one partition/task, limiting parallelism.

Picture it this way

Ten checkout lanes do not help if 70% of customers are forced into the same lane because they share one loyalty code.

Start from zero

- Skew means uneven data/work distribution across partitions/tasks, often driven by hot keys.
- Symptoms: one/few tasks much slower, much larger shuffle read, spills, long tail.
- Profile key frequencies before choosing a remedy.
- Possible remedies: broadcast small side, salting hot keys, two-stage aggregation, isolate hot keys, AQE skew join support.
- Salting complicates logic and requires unsalting/reaggregation; do not use automatically.
- Skew can exist even with many partitions.

Teacher demonstration

```
freq=(df.groupBy("customer_id").count().orderBy(F.desc("count")))
freq.show(10)
# Compare top-key share to total rows.
```

Read the example like English

- Key frequency makes skew concrete.
- If one key dominates, repartition(customer_id) will still send that key to one partition under hash partitioning.

Expected check

symptom	evidence
hot key	frequency distribution
slow tail	task duration metrics
large partition	shuffle/input bytes
remedy	must preserve correctness

Common beginner traps

- Adding more partitions and expecting one hot key to split automatically.
- Salting without validating final aggregates/joins.
- Calling any slow task skew without evidence.

Now you do a similar task

Run skew lab, identify hottest key/share and explain two possible remedies with correctness tradeoffs.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does repartition(100, "hot_key") not split one identical key across 100 partitions?

Answer in your own words before continuing.

DE12-12 - Adaptive Query Execution (AQE)

What you will be able to do

Understand AQE as runtime plan adaptation based on observed shuffle statistics, not a substitute for correct modeling.

Why this matters

Static estimates can be wrong. Spark can adapt join/shuffle choices after seeing actual intermediate data.

Picture it this way

AQE is a route planner that recalculates after seeing real traffic, but it still cannot fix a wrong destination.

Start from zero

- Adaptive Query Execution can coalesce shuffle partitions and adjust some join/skew strategies at runtime when enabled.
- AQE decisions occur after runtime statistics become available for query stages.
- It may switch to broadcast or split skewed partitions depending Spark version/config/query.
- Inspect final adaptive plan/runtime metrics, not only initial plan.
- AQE does not fix wrong join keys/grain or unsafe collect/UDF logic.
- Keep configuration explicit in performance evidence.

Teacher demonstration

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
result = ...
result.explain("formatted")
# Execute action, then inspect final plan/UI if available.
```

Read the example like English

- The initial plan can differ from final adaptive plan.
- Training should focus on identifying what changed, not forcing an AQE feature.

Expected check

AQE can help	AQE cannot fix
shuffle partition sizing	wrong business grain
some skew joins	duplicate dimension key
runtime join choice	driver collect explosion

Common beginner traps

- Claiming AQE makes tuning unnecessary.
- Comparing plans without executing when final adaptive plan is needed.
- Enabling many configs without recording them.

Now you do a similar task

Enable/inspect AQE on one shuffle/join lab and write what changed or did not. Explain one problem AQE cannot solve.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does AQE need runtime statistics before it can adapt some decisions?

Answer in your own words before continuing.

DE12-13 - Caching and persistence

What you will be able to do

Cache/persist only reused expensive DataFrames and prove benefit/cleanup rather than caching everything.

Why this matters

Caching trades memory/disk for avoiding recomputation. Unnecessary cache consumes executor resources and can slow other work.

Picture it this way

Cache is keeping an expensive intermediate dish warm because you will serve it several times, not storing every ingredient forever.

Start from zero

- cache/persist marks a DataFrame for storage after computation; it is lazy until an action materializes it.
- Useful when the same expensive lineage is reused by multiple actions and fits a sensible storage level.
- One-time DataFrames usually do not need cache.
- unpersist releases cached blocks when no longer needed.
- Measure reuse/runtime/plan/UI; do not claim benefit merely because cache() is present.
- Cache is not durable pipeline storage or checkpoint/recovery history.

Teacher demonstration

```
clean = expensive_transform(df).cache()
print(clean.count())          # materializes cache
print(clean.groupBy("region").count().count())
clean.unpersist()
```

Read the example like English

- First action computes and stores partitions.
- Second action can reuse cached clean data.
- unpersist frees storage afterwards.

Expected check

good cache candidate	bad candidate
expensive reused transform	single-use simple filter
iterative/repeated analysis	huge dataset larger than resources without plan
session reuse	durable historical storage

Common beginner traps

- cache() on every DataFrame.
- Forgetting an action is needed to populate cache.
- Using cache as pipeline checkpoint after application ends.

Now you do a similar task

Run cache lab and compare two actions with/without cache on the tiny project (timing may be noisy). Explain whether you would cache it in production and why.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can cache make a workload worse?

Answer in your own words before continuing.

DE12-14 - Parquet I/O and partition pruning

What you will be able to do

Read/write Parquet in Spark with explicit partitioning and identify partition pruning/projection in plans.

Why this matters

Spark commonly processes lake data; file layout can reduce scan work before distributed transformations begin.

Picture it this way

Partition directories are coarse shelves; Parquet column metadata is finer labels inside boxes. Spark can skip both when the query allows.

Start from zero

- `spark.read.parquet` preserves typed schema.
- select projects columns; filter predicates may be pushed to Parquet readers.
- Partitioned writes create directory keys such as `order_date=...`; filters on partition column can prune directories.
- Avoid over-partitioning/high-cardinality keys and tiny files.
- Inspect FileScan plan fields such as `PartitionFilters/PushedFilters/ReadSchema`.
- Reconcile source vs written/read counts and partitions; output file count matters operationally.

Teacher demonstration

```
df.write.mode("overwrite").partitionBy("region").parquet("output/orders")
west=(spark.read.parquet("output/orders")
      .filter(F.col("region")==="West")
      .select("order_id", "qty", "unit_price"))
west.explain("formatted")
```

Read the example like English

- region is encoded as partition directories.
- Filter on region can become `PartitionFilters`.
- Projection should narrow `ReadSchema`.

Expected check

plan evidence	meaning
PartitionFilters	directory pruning
PushedFilters	reader predicate pushdown
ReadSchema	projected columns
output part files	partition/task layout

Common beginner traps

- Partitioning by unique `order_id`.
- `coalesce(1)` large production writes.
- Saying pruning happened without looking at plan.

Now you do a similar task

Run Parquet I/O lab, save directory tree + filtered plan and prove the filtered result matches source truth.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What is the difference between partition pruning and Parquet predicate pushdown?

Answer in your own words before continuing.

DE12-15 - Built-in functions vs Python UDFs

What you will be able to do

Prefer Spark built-in expressions over Python UDFs when the logic can be expressed natively.

Why this matters

Built-in functions are visible to Spark optimizer and execute in Spark/JVM/vectorized paths; Python UDFs add serialization/runtime barriers.

Picture it this way

Built-ins are instructions the factory understands natively; a Python UDF sends each value to a separate interpreter booth.

Start from zero

- Use functions col, when, regexp_extract, split, lower, date functions, SQL expressions before custom Python UDF.
- Python UDFs can add serialization overhead and reduce optimizer visibility.
- Pandas UDFs/vectorized approaches can improve some custom workloads but still require justification.
- Correctness/readability matters: a simple UDF may be acceptable if no safe built-in exists, but measure.
- Never use a UDF to perform external network calls per row.
- Compare plan/runtime for equivalent native vs UDF logic when teaching performance.

Teacher demonstration

```
native = df.withColumn("amount_bucket",
    F.when(F.col("amount") >= 1000, "high").otherwise("normal"))
# Prefer this over a Python UDF for simple conditional logic.
```

Read the example like English

- Spark can see the native expression in its plan.
- No Python worker serialization is required for the simple conditional.

Expected check

logic	prefer
string/date/math/conditional	built-in functions
unsupported custom algorithm	consider UDF after evidence
external API per row	do not use UDF as network loop

Common beginner traps

- Writing UDF for lowercase/substring/date parsing.
- Calling external service inside UDF.
- Assuming Python UDF always unacceptable without profiling/use case.

Now you do a similar task

Rewrite one simple Python UDF-style transformation using built-ins and compare plan/output.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does optimizer visibility matter for built-in Spark expressions?

Answer in your own words before continuing.

DE12-16 - Driver safety: collect, toPandas and bounded inspection

What you will be able to do

Inspect distributed data safely without `collect()/toPandas()` accidentally pulling unbounded datasets into driver memory.

Why this matters

The driver is one process. Distributed scale disappears if you copy all rows back to it.

Picture it this way

A warehouse has many workers, but `collect` asks them to carry every box into the manager's office.

Start from zero

- `collect` returns all rows to driver memory; use only when result is provably small.
- `toPandas` collects into pandas/driver memory and is dangerous for unbounded/large DataFrames.
- `show/take/head/limit` can provide bounded inspection, but understand actions and ordering.
- For summaries use distributed aggregate then collect the tiny result.
- Write distributed output to storage rather than collect for export.
- Guard driver-side conversions with count/size limits in utilities and code review.

Teacher demonstration

```
# safer inspection
sample = df.limit(20)
sample.show(truncate=False)
summary = df.groupBy("region").agg(F.sum("gross").alias("sales"))
rows = summary.collect() # only if region result is known tiny
```

Read the example like English

- `limit` bounds the inspection DataFrame logically.
- Aggregation reduces distributed data before small collect.
- The unsafe lab intentionally shows why raw `collect` is a code smell for unknown volume.

Expected check

operation	driver risk
<code>df.collect()</code> unknown size	high
<code>df.toPandas()</code> unknown size	high
<code>df.limit(20).collect()</code>	bounded
small aggregate collect	bounded by known group cardinality

Common beginner traps

- Calling `collect` just to write CSV.
- `toPandas` on production fact table.
- Assuming `show` is free/no action.

Now you do a similar task

Complete the `bad_collect` TODO by replacing an unbounded driver pull with bounded inspection/distributed output. Explain your bound.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is "it worked on the sample" not evidence `collect()` is safe in production?

Answer in your own words before continuing.

DE12-17 - Testing and debugging Spark transformations

What you will be able to do

Write testable Spark transformations as pure DataFrame functions and debug schema/count/plan differences with small controlled fixtures.

Why this matters

Distributed jobs are hard to debug if business logic is mixed with session creation, I/O and global state.

Picture it this way

A testable transform is a machine component you can place on a small bench with known input/output before installing it in the factory.

Start from zero

- Write functions `def transform(df,...)->DataFrame` with no hidden read/write where practical.
- Create tiny DataFrame fixtures with explicit schema covering normal, null/boundary/duplicate cases.
- Assert row/key/measure results, not only that code runs.
- Use `printSchema/show/explain` on failing small fixture to isolate logic.
- Integration tests cover I/O/session; unit-like tests cover deterministic transform logic.
- Stop Spark sessions/fixtures cleanly and keep tests laptop-sized.

Teacher demonstration

```
def add_gross(df):  
    return df.withColumn("gross_sales", F.col("qty")*F.col("unit_price"))  
out=add_gross(test_df)  
assert out.orderBy("order_id").collect()[0]["gross_sales"] == 200.0
```

Read the example like English

- Function receives/returns DataFrame, so logic is reusable.
- Fixture is intentionally tiny, making bounded collect safe in a test.
- Production pipeline I/O is tested separately.

Expected check

test	purpose
schema fixture	type/null behavior
key uniqueness/count	grain
measure exact total	business correctness
explain on failure	plan diagnosis

Common beginner traps

- Testing only the full pipeline manually.
- Using huge production extracts in unit tests.
- Tests depending on row order without explicit `orderBy`.

Now you do a similar task

Implement the supplied testable transform TODO and at least three fixtures including a boundary/invalid case. Run tests and save failure then pass evidence.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can collect() be acceptable in a tiny deterministic unit test but dangerous in a production pipeline?

Answer in your own words before continuing.

DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint

What you will be able to do

Build and defend an integrated PySpark pipeline with explicit schema, correct joins/aggregations, controlled shuffles, Parquet output and driver-safe validation.

Why this matters

The final skill is not syntax; it is explaining distributed correctness and performance decisions together.

Picture it this way

You are planning a multi-worker factory line: correct input blueprint, balanced work, safe joins, bounded manager inspection and validated packaged output.

Start from zero

- Read source data with explicit schemas/quality rules.
- Filter/transform with built-ins and declared grain.
- Validate join cardinality before choosing broadcast/shuffle strategy.
- Reconcile source/valid rows and sales totals.
- Inspect plans for major shuffles/broadcast/pruning; optimize only with evidence.
- Write Parquet with sensible partitions/file counts for the scenario.
- Keep driver-safe inspection and tests.
- Run exact same pipeline again and prove deterministic output/control totals.

Teacher demonstration

```
# Project control totals from reference validation
source_rows = 18
completed_rows = 16
total_gross_sales = 25230.0
region_sales = {"East":5950,"North":6380,"South":6000,"West":6900}
category_sales = {"Accessories":16830,"Office":8400}
```

Read the example like English

- These totals come from independent source validation, not Spark reference solution.
- Your TODO implementation must reproduce them with correct grain/joins.
- A second run should produce the same controls.

Expected check

control	expected
source rows	18
valid completed rows	16
gross sales	25230.0
region/category totals	reconcile to 25230

Common beginner traps

- Opening pipeline_reference.py before first attempt.
- Optimizing partition counts while join fan-out is wrong.
- collecting full source to validate.

Now you do a similar task

Complete pipeline_TODO.py and tests closed-reference. Save controls, join checks, explain snippets and output file layout.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What evidence shows your Spark result is correct independently of whether it ran quickly?

Answer in your own words before continuing.